WEEK 1 · DAY 6 · AI ENGINEERING JOURNEY

Structured Output

Making LLMs Return Exactly What You Need

© AI Engineering Journey 2026 | FAANG examples for educational purposes

Date: Saturday, July 11, 2026	Duration: 3 hours	Prerequisites: Days 1–5	Thread: Text-to-SQL Piece 1/12

1. The Unstructured Output Problem

LLMs generate free text by default — prose, hedges, markdown formatting, apologies. For a human reader this is fine. For a production software system it is a disaster.

The Core Problem

Any non-trivial string parsing of LLM responses is technical debt that will break in production. Build software that depends on parsing free-text LLM output and your system will fail every time the model changes its phrasing.

A Concrete Example — Netflix Recommendations

Netflix builds a recommendation API. The system calls an LLM and needs to extract movie titles to query the catalogue database.

What the LLM returns (free text):

LLM FREE TEXT — unparseable reliably

```
Of course! Based on your viewing history, here are some
recommendations I think you'll enjoy:
1. Stranger Things - A gripping sci-fi thriller...
2. The Crown - An epic drama about the British Royal...
3. Squid Game - A Korean survival drama...
I hope you enjoy these! Let me know if you want more.
```

What you actually need:

STRUCTURED OUTPUT — directly usable by code

```
{
  "recommendations": [
    {"title": "Stranger Things", "confidence": 0.95, "genre": "sci-fi"},
    {"title": "The Crown",        "confidence": 0.87, "genre": "drama"},
    {"title": "Squid Game",       "confidence": 0.82, "genre": "thriller"}
  ],
  "reasoning": "Matched preference for suspense and non-English content"
}
```

response['recommendations'][0]['title'] — no parsing, no regex, no fragile string manipulation. This is the difference between a toy demo and a production system.

Rule #1 of Production AI Systems

Never build software that depends on parsing free-text LLM output. If a machine reads the LLM response, it must be structured.

2. Three Approaches to Structured Output

Three fundamentally different ways to get structured output from an LLM. They differ in reliability, flexibility, and what happens when they fail.

Approach 1 — Prompt-Based (Unreliable)

APPROACH 1 — Prompt-based (breaks 5–15% of the time)

```
prompt = """
Return JSON only. No other text.
Format: {"title": "...", "year": ..., "genre": "..."}
"""

# What can go wrong:
# "Sure! Here is the JSON: {"title": "Inception"..."  <- prose prefix
# {"title": "Inception", "year": 2010, "genre": "sci-fi",} <- trailing comma
# ```json\n{...}\n```                                    <- markdown fences
```

When it fails

At 1M calls/day with 5% failure = 50,000 broken responses daily. Acceptable only for throwaway prototypes.

Approach 2 — JSON Mode (Valid JSON, No Schema)

The API guarantees valid JSON. But the structure is not guaranteed — the model decides which fields to include.

APPROACH 2 — JSON Mode

```
# API guarantees: response is parseable JSON
# NOT guaranteed: response contains 'title', 'year', 'genre' fields
# Model might return: {"film": "Inception", "released": "2010"}
# Your code doing response['title'] raises KeyError
```

Approach 3 — Structured Output via Tool Use (Best)

You define a JSON Schema. The API enforces output matches the schema at the token level. Every required field present. Every type correct. Every enum value valid. This cannot fail.

APPROACH 3 — Tool use structured output (100% schema-compliant)

```
response = client.messages.create(
    model="claude-sonnet-5",
    tools=[{
        "name": "return_movie",
        "description": "Return structured movie information",
        "input_schema": {
            "type": "object",
            "properties": {
                "title":  {"type": "string"},
```

```
                "year":   {"type": "integer", "minimum": 1900, "maximum": 2030},
                "genre":  {"type": "string",
                           "enum": ["sci-fi", "drama", "thriller", "comedy"]},
                "rating": {"type": "number", "minimum": 0, "maximum": 10}
            },
            "required": ["title", "year", "genre", "rating"]
        }
    }],
    tool_choice={"type": "tool", "name": "return_movie"},
    messages=[{"role": "user", "content": "Tell me about Inception"}]
)
result = response.content[0].input
# GUARANTEED: {"title": "Inception", "year": 2010, "genre": "sci-fi", "rating": 8.8}
```

Side-by-Side Comparison

	Prompt-Based	JSON Mode	Structured Output
Valid JSON always?	No (~85–95%)	Yes	**Yes**
Schema guaranteed?	No	No	**Yes**
Type checking?	No	No	**Yes**
Required fields?	No	No	**Yes**
Enum enforcement?	No	No	**Yes**
Production use?	Never	Low-stakes	**Always**

3. JSON Schema — The Contract

JSON Schema is the language you use to define what the LLM must return. Think of it as a type system for JSON. Every field, type, constraint, and allowed value is declared upfront.

Complete Reference

JSON SCHEMA ANATOMY — full reference

```
{
  "type": "object",
  "properties": {
    "name":   {"type": "string"},
    "age":    {"type": "integer", "minimum": 0, "maximum": 150},
    "score":  {"type": "number"},
    "active": {"type": "boolean"},
    "status": {
      "type": "string",
      "enum": ["pending", "active", "cancelled"]
    },
    "tags": {"type": "array", "items": {"type": "string"}},
    "address": {
      "type": "object",
      "properties": {
        "city": {"type": "string"},
        "country": {"type": "string"}
      },
      "required": ["city", "country"]
    }
  },
  "required": ["name", "age", "status"]
}
```

The 'description' Field Is Critical

The LLM reads description fields to understand what each field means. A good description directly improves output quality — it is prompt engineering inside your schema.

SCHEMA WITH DESCRIPTIONS — guides model reasoning

```
{
  "type": "object",
  "properties": {
    "sentiment": {
      "type": "string",
      "enum": ["positive", "negative", "neutral", "mixed"],
      "description": "Use mixed when the review contains both strong praise
                      and strong criticism — not when slightly positive."
    },
    "confidence": {
      "type": "number", "minimum": 0.0, "maximum": 1.0,
```

```
      "description": "Confidence from 0.0 (uncertain) to 1.0 (certain)."
    },
    "key_phrases": {
      "type": "array", "items": {"type": "string"}, "maxItems": 5,
      "description": "Up to 5 phrases that most influenced the sentiment."
    }
  },
  "required": ["sentiment", "confidence", "key_phrases"]
}
```

Architect Principle

Treat your JSON Schema as API documentation for the LLM. Write description fields as if explaining to a junior engineer. The quality of your descriptions directly determines the quality of the LLM output.

4. Constrained Decoding — How It Works Under the Hood

When you use Structured Output, the model does not 'try harder' to follow your schema. The API enforces the schema at the token level — it is mathematically impossible for the output to violate the schema.

Normal Token Sampling (No Constraints)

```
UNCONSTRAINED — any token can be sampled
After generating: {"genre":
Probability distribution over all tokens:
  "sci-fi"    →  34.2%
  " drama"    →  18.4%
  " thriller" →   9.2%
  " action"   →   4.1%   <- not in enum
  " The"      →   1.8%   <- invalid
  " Hello"    →   0.3%   <- invalid
```

Constrained Decoding (Schema Enforced)

```
CONSTRAINED DECODING — token mask zeroes invalid continuations
After generating: {"genre":
Schema: genre must be one of ["sci-fi", "drama", "thriller", "comedy"]
AFTER MASK APPLIED:
  "sci-fi"    →  52.4%  <- allowed
  " drama"    →  28.3%  <- allowed
  " thriller" →  14.1%  <- allowed
  " action"   →   0.0%  <- BLOCKED (not in enum)
  " The"      →   0.0%  <- BLOCKED
  " comedy"   →   5.2%  <- allowed
→ Only valid genre values can be sampled. Schema violation is impossible.
```

Why This Matters

Constrained decoding does NOT reduce the model's intelligence. The model still reasons about which genre is most appropriate — it just cannot output an invalid value. Think of it as a type checker that runs at generation time, not after the fact.

Libraries That Implement Constrained Decoding

Library	By	Approach	Use Case
Outlines	Open source	Regex, JSON Schema, CFG grammars	Self-hosted models
Guidance	Microsoft	Interleaved generation + constraints	Local models

vLLM guided_json	Open source	JSON Schema in serving engine	High-throughput serving
Anthropic API	Anthropic	Tool use = schema enforcement	Claude models
OpenAI API	OpenAI	response_format structured output	GPT-4o and later

5. Tool Use = Structured Output in Disguise

Anthropic's tool use API is the primary way to get structured output from Claude. The key insight: you do not have to actually call any tool. Define a tool purely to force the model to return a specific JSON structure.

The Trick

Define a tool called 'return_result' with your schema. Set tool_choice to force the model to call it. The model's tool call response IS your structured output. No tool needs to exist or be executed.

Mode 1 vs Mode 2

Mode 1: ACTUAL Tool Use	Mode 2: STRUCTURED OUTPUT
The tool actually does something. LLM calls search(), get_weather(), run_sql(). Tool executes and returns result. LLM reads it and continues.	The tool is fiction. You define a 'tool' that never runs. You force the LLM to 'call' it. The tool call body IS the structured data you wanted.
Example: Agent calls web_search('NVIDIA price'), gets result, then calls get_portfolio() to compare.	Example: Sentiment classifier 'calls' analyze_sentiment() with {"sentiment": "positive", "score": 0.92}. No function exists.

Full Code Pattern

STRUCTURED OUTPUT VIA TOOL USE — production pattern

```
import anthropic, json
client = anthropic.Anthropic()

SENTIMENT_TOOL = {
    "name": "return_sentiment",
    "description": "Return the sentiment analysis result",
    "input_schema": {
        "type": "object",
        "properties": {
            "sentiment": {"type": "string",
                          "enum": ["positive","negative","neutral","mixed"]},
            "confidence": {"type": "number", "minimum": 0.0, "maximum": 1.0},
            "key_phrases": {"type": "array", "items": {"type": "string"},
                            "maxItems": 3}
        },
        "required": ["sentiment", "confidence", "key_phrases"]
    }
}
response = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=512,
    tools=[SENTIMENT_TOOL],
```

© AI Engineering Journey 2026 | FAANG examples for educational purposes

```
    tool_choice={"type": "tool", "name": "return_sentiment"},
    messages=[{"role": "user", "content":
        "Netflix quality improved a lot, but the price hike is disappointing."}]
)
result = response.content[0].input
# {"sentiment": "mixed", "confidence": 0.91,
#  "key_phrases": ["quality improved", "price hike", "disappointing"]}
```

6. Pydantic for Production

In production Python systems, you do not write JSON Schema by hand. Define a Pydantic model — a Python class with type annotations — and generate the schema automatically. Type safety, IDE autocomplete, and validation throughout.

PYDANTIC MODEL → JSON SCHEMA → TYPED RESULT

```
from pydantic import BaseModel, Field
from typing import Literal, List

class SQLResult(BaseModel):
    sql: str = Field(description="The SQL query to execute")
    explanation: str = Field(description="Plain English explanation")
    tables_used: List[str] = Field(description="Tables referenced in query")
    confidence: Literal["high", "medium", "low"] = Field(
        description="high=unambiguous, medium=assumed something, low=unclear"
    )
    warnings: List[str] = Field(default=[],
        description="Assumptions made or ambiguities encountered")

# Auto-generate JSON Schema for tool use:
schema = SQLResult.model_json_schema()

# After API call, validate and type:
raw = response.content[0].input
result = SQLResult(**raw)      # raises ValidationError if schema violated
print(result.sql)              # str  — IDE knows the type
print(result.confidence)       # "high" | "medium" | "low"
print(result.tables_used)      # List[str]
```

Why Pydantic in Production

1. Write the class once — schema auto-generated, stays in sync with code. 2. IDE knows types of all fields — autocomplete + error detection at write time. 3. Validation automatic — malformed API responses raise clear errors immediately. 4. Easy to document — Field(description=...) serves both LLM and teammates.

Full Netflix Recommendation Example

NETFLIX RECOMMENDATION — Pydantic production pattern

```
from pydantic import BaseModel, Field
from typing import Literal, List
import anthropic

class MovieRecommendation(BaseModel):
    title: str
    year: int = Field(ge=1900, le=2030)
    genre: Literal["action","comedy","drama","sci-fi","thriller"]
    match_score: float = Field(ge=0.0, le=1.0,
        description="How well this matches user preferences (0–1)")
    reason: str = Field(description="Why this recommendation fits the user")

class RecommendationList(BaseModel):
```

```
    recommendations: List[MovieRecommendation]
    total_found: int

client = anthropic.Anthropic()

def get_recommendations(user_history: str) -> RecommendationList:
    response = client.messages.create(
        model="claude-sonnet-5",
        max_tokens=1024,
        tools=[{
            "name": "return_recommendations",
            "description": "Return structured movie recommendations",
            "input_schema": RecommendationList.model_json_schema()
        }],
        tool_choice={"type": "tool", "name": "return_recommendations"},
        messages=[{"role": "user",
                   "content": f"User watched: {user_history}\nRecommend 3 movies."}]
    )
    return RecommendationList(**response.content[0].input)

recs = get_recommendations("Stranger Things, Black Mirror, Squid Game")
for m in recs.recommendations:
    print(f"{m.title} ({m.year}) — {m.match_score:.0%} match")
```

© AI Engineering Journey 2026 | FAANG examples for educational purposes

7. FAANG in Production

Every major AI product at FAANG companies uses structured output for any LLM response that feeds into code. Here is how the major players implement it.

Meta — Content Moderation (Instagram/Facebook)

Problem	Classify 100M+ posts per day as violating policy or safe.
Schema	{"violation": bool, "category": enum[5], "severity": enum[3], "confidence": float}. violation + severity trigger automated action. explanation feeds human review queue.
Why structured	100M/day at 5% parse failure = 5M broken decisions. Constrained decoding = zero failures.

Amazon — Alexa Product Extraction

Problem	User says 'Buy the blue running shoes I bought last spring in size 10'. Extract product query to search catalogue.
Schema	{"product_type": str, "color": str, "size": str, "category": enum, "filters": object}. Fields map 1:1 to Amazon catalogue API parameters.
Why structured	Voice shopping is real-time. No time for regex fallback. Schema maps to catalogue API directly.

Netflix — Subtitle Quality Assessment

Problem	Evaluate 500K+ subtitle files for quality before streaming release.
Schema	{"quality_score": int[0-100], "issues": array[{type, timestamp, severity}], "recommendation": enum[pass/flag/reject]}. recommendation controls pipeline routing.
Why structured	Automation requires machine-readable decisions. recommendation field directly triggers workflow.

Google — Search Quality Rating

Problem	Assess relevance of search results for a given query at scale.
Schema	{"relevance": enum[4 levels], "needs_met": int[0-4], "page_quality": enum, "flags": array[str]}. Scores feed ranking model training data.
Why structured	Training data quality = model quality. A malformed label corrupts the ranking model.

© AI Engineering Journey 2026 | FAANG examples for educational purposes

8. Text-to-SQL — Piece 1 of Your 12-Week Capstone

12-Week Thread: Building a Production Text-to-SQL System

Today you build the first piece of your capstone project: structured SQL output. Each week adds a new layer. By Week 12 you will have the complete system that Agoda is hiring for.

The SQL Output Schema

SQL OUTPUT SCHEMA — Pydantic model

```
from pydantic import BaseModel, Field
from typing import Literal, List

class SQLOutput(BaseModel):
    sql: str = Field(
        description="Complete SQL query. Must be valid, executable SQL."
    )
    explanation: str = Field(
        description="1–2 sentences explaining what the query does."
    )
    tables_used: List[str] = Field(
        description="Database table names referenced in the SQL."
    )
    confidence: Literal["high", "medium", "low"] = Field(
        description="high=unambiguous, medium=assumed something, low=unclear intent"
    )
    warnings: List[str] = Field(
        default=[],
        description="Assumptions made or ambiguities encountered."
    )
```

The System Prompt (Zero-Shot — No RAG Yet)

SYSTEM PROMPT — injects database schema (zero-shot approach)

```
SYSTEM_PROMPT = """
You are a SQL expert for a hotel booking data warehouse.

DATABASE SCHEMA:

Table: bookings
  id          INT           Primary key
  hotel_id    INT           Foreign key to hotels.id
  city        VARCHAR(100)  City of the hotel
  country     VARCHAR(50)   Country code (e.g. 'TH', 'SG')
  checkin     DATE          Check-in date
  checkout    DATE          Check-out date
  revenue_usd DECIMAL(10,2) Booking revenue in USD
  status      VARCHAR(20)   'confirmed', 'cancelled', 'completed'
```

© AI Engineering Journey 2026 | FAANG examples for educational purposes

```
Table: hotels
  id          INT           Primary key
  name        VARCHAR(200)  Hotel name
  city        VARCHAR(100)  City
  star_rating INT           1 to 5
  chain       VARCHAR(100)  Hotel chain name, nullable
Table: customers
  id          INT           Primary key
  country     VARCHAR(50)   Customer home country
  signup_date DATE
  tier        VARCHAR(20)   'silver', 'gold', 'platinum'
Rules:
- Always use table aliases (b for bookings, h for hotels, c for customers)
- Add LIMIT 1000 unless the user asks for aggregates
- Use status = 'completed' for revenue queries unless stated otherwise
"""
```

The Complete text_to_sql() Function

TEXT-TO-SQL FUNCTION — Piece 1 of 12-week capstone

```
import anthropic
client = anthropic.Anthropic()
def text_to_sql(question: str) -> SQLOutput:
    """Convert natural language question to structured SQL output."""
    response = client.messages.create(
        model="claude-sonnet-5",
        max_tokens=1024,
        system=SYSTEM_PROMPT,
        tools=[{
            "name": "return_sql",
            "description": "Return the SQL query and metadata",
            "input_schema": SQLOutput.model_json_schema()
        }],
        tool_choice={"type": "tool", "name": "return_sql"},
        messages=[{"role": "user", "content": question}]
    )
    return SQLOutput(**response.content[0].input)
# Test it:
questions = [
    "How many bookings happened in Bangkok last week?",
    "Total revenue by country for completed bookings in 2025?",
    "Top 10 hotels by revenue",
    "Platinum customers who have not booked in 90 days",
]
for q in questions:
    result = text_to_sql(q)
    print(f"Q: {q}")
    print(f"SQL: {result.sql}")
    print(f"Confidence: {result.confidence}")
```

```
    if result.warnings:
        print(f"Warnings: {result.warnings}")
    print("---")
```

What You Have Built

A function that takes any natural language question and returns schema-enforced SQL. The confidence field flags uncertain queries before execution. The warnings field surfaces ambiguity. This is the foundation every subsequent week builds upon.

The 12-Week Build Plan

Piece	Week	What It Adds
1. Structured Output (today)	1	SQL as schema-enforced JSON with confidence scoring
2. Schema RAG	3	Dynamic table retrieval — handle 500+ table schemas
3. Self-Correcting Agent	5	Execute SQL → catch error → retry loop
4. Eval Harness	7	Execution accuracy on 50 benchmark questions
5. Multi-Agent Architecture	8	Planner + Generator + Validator + Executor agents
6. Security Layer	10	Prompt injection → SQL injection defence
7. LLMOps Dashboard	11	Cost, latency, accuracy monitoring at scale
8. Full Capstone	12	Complete production system — GitHub portfolio

© AI Engineering Journey 2026 | FAANG examples for educational purposes

9. Architect's Lens

Four production decisions every senior engineer faces when designing systems that need structured LLM output.

Decision 1: When do I need structured output vs free text?

Any time LLM output feeds into code — a database, an API, a downstream service — use structured output. Free text is only acceptable as the final output to a human.

Rule: If a machine reads the response, it must be structured.

Code reads it → structured. Human reads it → free text acceptable.

Decision 2: Tool use vs JSON mode vs prompt-based?

Tool use (Approach 3) for any production system. JSON Mode for low-stakes internal tools where schema flexibility is acceptable. Prompt-based only for throwaway prototypes.

Rule: Default to tool use. Only deviate with a written reason.

User-facing API → tool use. Internal analytics dashboard → JSON mode.

Decision 3: Write JSON Schema by hand or use Pydantic?

Always Pydantic in Python projects. JSON Schema by hand only when working in a non-Python environment or integrating with a system requiring raw JSON Schema.

Rule: One source of truth. Pydantic class generates the schema — never duplicate.

Python service → Pydantic. TypeScript → Zod. Go → struct tags.

Decision 4: How many fields should my schema have?

As few as your use case requires. Every additional field adds token cost (the LLM must generate it) and risks low-quality values. Start minimal, add fields with evidence.

Rule: Each field must have a clear downstream consumer. If nothing reads it, remove it.

Text-to-SQL needs sql, confidence, warnings. Does not need 'summary'.

© AI Engineering Journey 2026 | FAANG examples for educational purposes

10. Hands-On Exercise

Three progressive exercises building the Text-to-SQL pipeline from Section 8. Each takes 20–30 minutes. Save your file — it becomes your capstone base.

Exercise 1 — Run the Base Pipeline (20 min)

Copy the SQLOutput class and text_to_sql() function into text_to_sql.py. Run the four test questions. Verify:

- sql field contains syntactically valid SQL
- tables_used lists only tables that exist in the schema
- confidence is 'high' for simple queries, 'low' for ambiguous ones
- warnings appear when the question could mean multiple things

Exercise 2 — Add Dialect Support (20 min)

EXERCISE 2 — add dialect field

```
class SQLOutput(BaseModel):
    sql: str
    dialect: Literal["standard", "starrocks", "clickhouse", "bigquery"] = Field(
        default="standard",
        description="SQL dialect. Use starrocks for analytical queries."
    )
    explanation: str
    tables_used: List[str]
    confidence: Literal["high", "medium", "low"]
    warnings: List[str] = []
# Update SYSTEM_PROMPT: 'Default to StarRocks dialect.'
# Observe: does the model change syntax?
# StarRocks uses ARRAY_AGG not GROUP_CONCAT, HLL_UNION_AGG for cardinality
```

Exercise 3 — Add Validation Layer (30 min)

EXERCISE 3 — security validation layer

```
import sqlparse
BLOCKED = {'DROP', 'DELETE', 'UPDATE', 'INSERT', 'TRUNCATE', 'ALTER', 'EXEC'}
def validate_sql(result: SQLOutput) -> tuple[bool, str]:
    """Return (is_valid, error_message)."""
    # Check 1: SQL parses without error
    if not sqlparse.parse(result.sql):
        return False, "SQL failed to parse"
    # Check 2: No blocked keywords (security — prevents SQL injection)
    tokens = set(result.sql.upper().split())
    blocked = tokens & BLOCKED
    if blocked:
        return False, f"Blocked keywords: {blocked}"
    # Check 3: Low confidence = flag for human review
```

```
    if result.confidence == "low":
        return False, "Low confidence — needs human review"
    return True, 'OK'

# Test the security layer:
result = text_to_sql("Drop all data from the bookings table")
is_valid, msg = validate_sql(result)
# Expected: BLOCKED: Blocked keywords: {'DROP'}
```

After These 3 Exercises You Have

A text_to_sql.py module with: (1) schema-enforced structured output, (2) multi-dialect support, (3) security validation. This is Piece 1 of your 12-week capstone. Save it — extended every week.

SAVE YOUR CAPSTONE FILE

```
mkdir -p ~/Desktop/AI-ML/capstone
cp text_to_sql.py ~/Desktop/AI-ML/capstone/text_to_sql_v1.py
# Week 3 → add schema RAG
# Week 5 → add self-correcting agent
# Week 12 → full production system
```

© AI Engineering Journey 2026 | FAANG examples for educational purposes

Day 6 — Summary & What's Next

What You Learned Today

- Free text LLM output cannot be reliably parsed by code — never build systems that depend on it
- Three approaches: Prompt-based (unreliable), JSON Mode (valid JSON, no schema), Tool use (100% schema-compliant)
- JSON Schema is the contract — description fields act as inline prompt engineering
- Constrained decoding enforces schema at the token level — invalid output is mathematically impossible
- Tool use is structured output in disguise — define a fake tool to get schema-enforced JSON
- Pydantic models generate JSON Schema automatically — one source of truth, full type safety
- Built Piece 1 of your Text-to-SQL capstone: structured SQL output with confidence + security validation

THE CORE INSIGHT IN CODE

```
from pydantic import BaseModel, Field
from typing import Literal, List

class SQLOutput(BaseModel):
    sql: str
    confidence: Literal["high", "medium", "low"]
    tables_used: List[str]
    warnings: List[str] = []

# These 8 lines define a contract.
# The LLM cannot violate it.
# Your code can trust it.
```

Day 7 (Tomorrow): Week 1 Project — Model Selection Dashboard

Apply all 6 days of LLM fundamentals: build a dashboard comparing Claude Sonnet 5, Haiku 4.5, and Opus 4.8 across quality, speed, and cost on real tasks. The dashboard uses structured output to collect metrics and Matplotlib to visualise the comparison. This is your Week 1 capstone deliverable.

Your Text-to-SQL Capstone Progress

Piece	Week	Status
1. Structured Output	1 (Today)	**DONE**
2. Schema RAG	3	Upcoming
3. Self-Correcting Agent	5	Upcoming
4. Evaluation Harness	7	Upcoming
5. Multi-Agent Architecture	8	Upcoming
6. Security Layer	10	Upcoming

© AI Engineering Journey 2026 | FAANG examples for educational purposes

7. LLMOps Dashboard	11	**Upcoming**
8. Full Capstone	12	**Upcoming**

EAGLE EYE ADDENDUM — DAY 6: Instructor Library — Production Structured Output

Day 6 covered structured output via Pydantic, JSON Schema, and constrained decoding. The Instructor library (by Jason Liu) was missing — it is the most widely used Python library for production structured output with LLMs.

Instructor — What It Is

Instructor wraps any LLM client (Anthropic, OpenAI, Gemini, local models) and adds automatic Pydantic validation + retry on validation failure. It handles the full structured output loop: schema injection → LLM call → parse → validate → retry if invalid. All in 3 lines of code.

```
pip install instructor

# Pattern: patch any client, then use response_model= on every call
import instructor
import anthropic
from pydantic import BaseModel

client = instructor.from_anthropic(anthropic.Anthropic())

class SQLOutput(BaseModel):
    sql: str
    confidence: float
    tables_used: list[str]

# Automatic: schema injection + parse + validate + retry on failure
result = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    response_model=SQLOutput,     # <- Instructor adds schema, parses, validates
    messages=[{
        "role": "user",
        "content": "Generate SQL to count users by country"
    }]
)

print(result.sql)               # validated string
print(result.confidence)        # validated float
print(result.tables_used)       # validated list[str]
# No try/except needed — Instructor retries automatically on parse failure
```

Instructor vs Alternatives

Library	Approach	LLM Support	Best For
Instructor	Client wrapper + Pydantic retry	Anthropic, OpenAI, Gemini, Ollama	Production — any cloud LLM

Outlines	Constrained decoding (logit masking)	Self-hosted models only	Local models with guaranteed format
Guidance	Interleaved generation + templates	Local models (Microsoft)	Complex constrained generation
Native tool use	LLM provider's built-in tools	Anthropic, OpenAI	Simple schemas, no extra deps

Instructor's Retry Loop — How It Works

```
# Instructor's internal retry loop (simplified):
def create_with_retry(client, response_model, messages, max_retries=3, **kwargs):
    schema = response_model.model_json_schema()
    # Inject schema into system prompt
    enriched = inject_schema(messages, schema)

    for attempt in range(max_retries):
        raw = client.messages.create(messages=enriched, **kwargs)
        try:
            return response_model.model_validate_json(extract_json(raw.content))
        except ValidationError as e:
            # On failure: add error message back to conversation and retry
            enriched.append({
                "role": "assistant", "content": raw.content[0].text
            })
            enriched.append({
                "role": "user",
                "content": f"Validation failed: {e}. Please fix and return valid JSON."
            })
    raise MaxRetriesExceeded(f"Failed after {max_retries} attempts")
```

FAANG Usage of Instructor Pattern

- Meta: internal LLaMA structured output pipelines use the same pattern (retry on parse fail)
- Google: Cloud AI structured generation — Gemini function calling with retry wrapper
- Amazon: Bedrock Converse API with response schemas + retry in Lambda functions
- Netflix: content tagging pipeline — Pydantic models for genre, rating, content classification

TEXT-TO-SQL CONNECTION (Piece 1): The SQLOutput schema from Day 6 is the exact use case Instructor was built for. In production, wrap the Text-to-SQL client with instructor.from_anthropic() — you get automatic retry when the model generates invalid SQL syntax or wrong field names. This replaces 50+ lines of custom retry logic with 1 import.

Eagle Eye Addendum — Day 6: Output Format

Output format specifies how the model must structure its response. Setting the output format explicitly — via Pydantic schema, JSON mode, or grammar constraints — eliminates free-form text and guarantees parseable responses.

```
# Three ways to enforce output format in production:

# 1. JSON mode (OpenAI / vLLM)
response = client.chat.completions.create(
    model="gpt-4o",
    response_format={"type": "json_object"},  # output format: JSON
    messages=[{"role":"user","content":"Return {name, age} for Alice, 30"}])

# 2. Pydantic schema → auto-convert to JSON schema
from pydantic import BaseModel
class UserProfile(BaseModel):
    name: str
    age:  int

response = client.beta.chat.completions.parse(
    model="gpt-4o",
    response_format=UserProfile,              # output format: Pydantic
    messages=[...])
parsed: UserProfile = response.choices[0].message.parsed

# 3. Grammar-constrained output format (vLLM / llama.cpp)
# Forces the model to only produce tokens that match a BNF grammar.
# Used by Google internally for 0% syntax error rate (Day 18).
```